WOLF

PROGRAMMING LANGUAGE

Builder PRD · v3.1 · 3-Month Sprint

Version	Date	Status
3.1 — TraversyMVC Edition	February 2026	**ACTIVE**

PHP familiarity · Go performance · Python ML · Config-driven databases

Team: Human Founder + Claude Opus 4.6 + Gemini 3

How To Use [redacted]cument

> *This is a builder's manual, not a vision document. Every section tells you what to build, how to build it, and how to know when it is done. Open it, pick the first TODO task in the current sprint week, and start writing code.*

This PRD has four parts:

- Part A — Architecture & Language Spec: Locked decisions. Read before writing a single line.
- Part B — Sprint Task Boards: 12 weeks of tasks with owners, deadlines, and acceptance criteria.
- Part C — Code Specification: Exact Go structs, interfaces, and data structures. The implementation contract.
- Part D — Testing & Launch: How to verify Wolf works and how to ship it.

Changelog from v3.0: (1) Database system redesigned to mirror TraversyMVC exactly — libraries/Database.wolf class, config/config.wolf constants, $this->db-> usage in models. No DB:: facade anywhere. (2) wolf.python environment management confirmed as first-class feature. (3) Repository structure updated with libraries/ and correct config/ role.

PART A — ARCHITECTURE & LANGUAGE SPECIFICATION

A1. What Wolf Is — The Locked Definition

> *These decisions do not change during the 3-month sprint. Any proposal to change them requires a written decision from the Founder before the team acts on it.*

A1.1 Wolf in One Sentence

Wolf is a compiled programming language with PHP-inspired syntax that transpiles to Go, uses a config-file-driven database layer, and embeds CPython for native ML library access.

A1.2 The Four Pillars

PHP syntax feel	Dollar-sign variables, familiar control flow, string interpolation, no mandatory semicolons. A PHP developer reads Wolf code without a manual.
Go performance	Wolf transpiles to idiomatic Go. The final artefact is a native binary compiled by gc. Users get goroutines, channels, and Go's memory model automatically.
TraversyMVC database	Database connection class lives in libraries/Database.wolf. Constants (DB_HOST, DB_USER etc) live in config/config.wolf. Models do new Database in constructor and use $this->db->query/bind/execute/single/resultSet. Identical to TraversyMVC PHP pattern.
Python ML bridge	@ml blocks embed CPython via cgo. Any pip-installable package works. wolf.python manages the virtual environment automatically. No pip or venv commands needed by the developer.

A1.3 What Wolf Is NOT in This Sprint

- NOT a new runtime — Wolf uses Go's runtime entirely.
- NOT a Python replacement — @ml calls into Python. Wolf itself is not Python.
- NOT wolfpkg (package manager) — this is Phase 2. Go modules used for dependencies in Phase 1.
- NOT a query builder or ORM — models use raw prepared statements via $this->db-> exactly like TraversyMVC.
- NOT statically typed by default — dynamic typing first, optional strict mode later.

A1.4 Compiler Pipeline — 7 Stages

#	Stage	Input → Output	Responsibility
1	**Lexer**	`Source → []Token`	Tokenise every character. Typed tokens with line+col. Handle $vars, @ml, string interpolation.
2	**Parser**	`[]Token → AST`	Recursive-descent PEG parser. Typed AST nodes. Syntax errors with line/col and helpful message.
3	**Resolver**	`AST → Annotated AST`	Resolve variable scopes. Strip $ prefix for Go output. Detect undeclared variables.
4	**Type Checker**	`Ann. AST → Typed AST`	Optional strict mode. Always: validate @ml boundary types. Validate DB config references.
5	**IR Emitter**	`Typed AST → WIR`	Wolf Intermediate Representation. Flatten closures, normalise interpolation, resolve DB config.
6	**Go Emitter**	`WIR → .go source`	Emit gofmt-compatible Go. @ml → cgo calls. async → goroutines. Database class → Go PDO-equivalent driver calls.
7	**Go Build**	`wolf_out/ → binary`	Shell to go build. Single native binary. Users never see generated .go files.

A1.5 Repository Structure

```
wolf/  cmd/wolf/              ← main.go — CLI entry (wolf build, run, fmt, python, test)
internal/    lexer/               ← lexer.go, token.go, token_types.go    parser/
← parser.go, ast.go, ast_nodes.go    resolver/            ← resolver.go, scope.go
typechecker/          ← checker.go, types.go    ir/                   ← ir.go, ir_nodes.go
goemitter/            ← emitter.go, helpers.go    mlbridge/            ← bridge.go,
cgo_bridge.c, cgo_bridge.h    pythonenv/           ← manager.go, config.go, lock.go
stdlib/               ← http.go, json.go, strings.go, math.go, env.go  examples/
hello/      hello.wolf    webserver/           ← a full example Wolf app (mirrors
TraversyMVC structure)      config/        config.wolf      ← DB_HOST, DB_USER, DB_PASS,
DB_NAME constants + app config      libraries/        Database.wolf    ← Wolf database
connection class (Wolf's PDO equivalent)      models/        User.wolf        ← uses new
Database, $this->db->query/bind/execute      controllers/        UserController.wolf
views/      wolf.python        ← Python requirements (if ML used)      .env
← actual credentials (NEVER committed to git)      .env.example       ← key template
committed to git      main.wolf    ml_demo/             ← surge_pricing.wolf (showcase
app)  docs/  go.mod  go.sum  README.md  Makefile  LICENSE
```

A1.6 Technology Stack

Compiler language	Go 1.22+. The compiler is a Go module. No exceptions.
Transpile target	Go 1.22+. Wolf emits Go source, calls go build.
ML bridge	cgo + libpython 3.11. Python 3.11 for stability and broad ML library support.
Python env mgmt	wolf.python config file + wolfpkg-managed venv. Developer never touches pip or venv directly.
HTTP server	Go net/http wrapped in Wolf-friendly API.

Database	libraries/Database.wolf class pattern. database/sql + go-sql-driver/mysql. go-redis/redis. mongo-driver. Constants from config/config.wolf. No DB:: facade anywhere.
CLI framework	cobra for wolf subcommands.
Testing	Go testing + testify/assert for unit tests. Wolf test files for integration tests.
Build/CI	Makefile + GitHub Actions. Every PR must pass tests before merge.
Version control	Git + GitHub. Branches: main (stable), dev (active), feature/* (tasks).

A2. Wolf Lan[redacted]pecificat[redacted]

> *This section is the source of truth. The parser must accept everything here and reject everything not here. When Claude or Gemini is unsure how something should work, the answer is in this section.*

A2.1 Variables

```
$name   = "Wolf"          // string — dynamic type default$count  = 0               //
integer$price  = 9.99            // float$active = true            // boolean$items  =
[1, 2, 3]       // array$user   = {"id": 1}      // map// Strict / static typing (opt-
in)var $userId:  int    = 42var $label:   string = "hello"var $scores:  []float64 =
[9.1, 8.7, 9.4]
```

A2.2 String Interpolation

```
$city  = "Lagos"$msg   = "Pickup in {$city}"                    // single variable$total
= "Total: ${$price * $qty} NGN"          // expression$info  = "Driver {$driver->name}
is {$driver->rating} stars"  // property
```

A2.3 Functions

```
func greet($name) {    return "Hello {$name}"}// Multiple return valuesfunc divide($a,
$b) -> (float, error) {    if $b == 0 { return 0, error("division by zero") }    return
$a / $b, nil}// Arrow shorthandfunc double($n) => $n * 2// Named parametersfunc
createUser(name: $name, email: $email, role: $role = "user") { ... }$user =
createUser(name: "Ada", email: "ada@wolf.dev")// First-class / closure$multiply =
func($a, $b) { return $a * $b }$result = $multiply(6, 7)  // 42
```

A2.4 Control Flow

```
// if / elseif $score > 90 { $grade = "A" }else if $score > 75 { $grade = "B" }else
{ $grade = "C" }// for (C-style)for $i = 0; $i < 10; $i++ { print($i) }// foreachforeach
$items as $item { print($item) }foreach $users as $id => $user { print("{$id}:
{$user->name}") }// whilewhile $running { $running = checkStatus() }// match
(exhaustive, compiles to Go switch)match $status {    "active"    => handleActive()
"pending"   => handlePending()    "cancelled" => handleCancel()    _            =>
handleDefault()}
```

A2.5 Classes

```
class Driver {    $name:   string    $rating: float    $online: bool = false    func
__construct($name, $rating) {        $this->name   = $name        $this->rating =
$rating    }    func goOnline()  { $this->online = true }    func goOffline()
{ $this->online = false }    func summary() -> string {        return "{$this->name} —
{$this->rating}★"    }}$driver = new Driver("Ade", 4.8)$driver->goOnline()
print($driver->summary())  // Ade — 4.8★
```

A2.6 Concurrency

```
// async — spawns goroutineasync { $result = heavyTask() }// await — wait for result
$task    = async { return fetchDrivers($location) }$drivers = await $task// parallel —
run multiple tasks, wait for ALL to finishparallel {    $drivers =
fetchDrivers($location)    $surge    = calculateSurge($zone)    $eta     =
estimateETA($origin, $dest)}// execution continues here only when all three complete//
channels (Go channels exposed directly)$ch = channel(int)async { send($ch,
computeResult()) }$value = receive($ch)
```

A2.7 Error Handling

```
// Go-style multiple returns[$data, $err] = parseInput($raw)if $err != nil { return
$res->error($err, 400) }// try/catch sugar (compiles to if err != nil)try {    $user =
fetchUser($id)} catch ($e) {    log("Failed: {$e}")    return $res->error("User not
found", 404)}
```

A2.8 The @ml Block

> *The @ml block is Wolf's most unique feature. Wolf variables in scope are automatically available as Python variables (without $). Python variables assigned inside are available back in Wolf after the block exits.*

```
// Basic @ml block@ml {    import numpy as np    result = np.array(items).mean()  //
'items' came from Wolf $items}print($result)  // available in Wolf after block//
Explicit in/out (recommended for clarity in large files)@ml(in: [$location, $rideCount],
out: [$price, $confidence]) {    from wolf_models import surge    price, confidence =
surge.predict(location, ride_count)}// Named cached model — model loaded once, reused
across requests@ml model("surge_v2") {    import torch    prediction =
model.forward(features)  // 'model' is the loaded cache}// Async @ml — ML runs in
background, does not block HTTP response$task = @ml async {    from transformers import
pipeline    sentiment = pipeline("sentiment-analysis")(reviewText)}$sentiment = await
$task
```

A2.8.1 @ml Type Bridge Rules

Wolf Type	Python Type	Direction	Notes
`int / float`	`int / float`	↔ Both ways	Direct numeric conversion
`string`	`str`	↔ Both ways	UTF-8, no truncation
`bool`	`bool`	↔ Both ways	true/false ↔ True/False
`array []T`	`list`	↔ Both ways	Elements must each be bridgeable
`map[string]any`	`dict`	↔ Both ways	Keys must be strings
`struct`	`dict`	Wolf → Python	Fields become dict keys
`[]byte`	`bytes`	↔ Both ways	Used for raw data and tensors
`nil`	`None`	↔ Both ways	
`func`	`NOT supported`	—	Cannot cross the boundary

channel	NOT supported	—	Cannot cross the boundary

A2.9 HTTP Server

```
$app = Wolf\App::create()// Middleware$app->use(Wolf\Middleware::cors())
$app->use(Wolf\Middleware::json())$app->use(Wolf\Middleware::auth($secret))// Routes
$app->get("/",                  func($req, $res) { return $res->json(["ok"=>true]) })
$app->post("/api/rides",        "RideController@create")$app->get("/api/rides/{id}",
"RideController@show")$app->put("/api/rides/{id}",   "RideController@update")
$app->delete("/api/rides/{id}","RideController@cancel")// Route groups
$app->group("/api/v1", func($r) {    $r->get("/users",  "UserController@index")
$r->post("/users", "UserController@store")})$app->serve(port: 8080, workers: "auto")  //
auto = GOMAXPROCS goroutines
```

A2.10 Database — TraversyMVC Pattern

> *FINAL DECISION — Wolf uses the exact TraversyMVC database pattern. No DB:: facade. No query builder. The Database class lives in libraries/Database.wolf. Constants live in config/config.wolf. Models instantiate new Database in their constructor and use $this->db->query/bind/execute/single/resultSet. Any PHP developer who has used TraversyMVC can write Wolf database code immediately.*

A2.10.1 Project Structure

Every Wolf web app follows this folder structure:

```
myapp/  config/    config.wolf      ← DB constants + app constants. Reads from .env.
libraries/    Database.wolf    ← Wolf database connection class  models/    User.wolf
← instantiates new Database, uses $this->db->    Ride.wolf  controllers/
UserController.wolf  views/  wolf.python        ← Python requirements (only if @ml
used)  .env               ← real credentials — NEVER committed to git  .env.example
← key template — committed to git  main.wolf
```

A2.10.2 config/config.wolf — Constants File

The config file reads from .env and defines constants the whole application uses. This replaces PHP's define() calls and the DB_HOST constant pattern exactly.

```
// config/config.wolf// Reads credentials from .env and exposes them as constants.//
This file IS committed to git. Real values are in .env only.# Databasedefine("DB_HOST",
Env::get("DB_HOST", "localhost"))define("DB_USER", Env::get("DB_USER", "root"))
define("DB_PASS", Env::get("DB_PASS", ""))define("DB_NAME", Env::get("DB_NAME",
"myapp"))define("DB_PORT", Env::get("DB_PORT", "3306"))# Applicationdefine("APP_NAME",
Env::get("APP_NAME",    "MyApp"))define("APP_VERSION", "1.0.0")define("BASE_URL",
Env::get("BASE_URL",    "http://localhost:8080"))define("DEBUG",
Env::get("APP_DEBUG",   "false") == "true")# JWT / Authdefine("JWT_SECRET",
Env::get("JWT_SECRET",  "change-me"))define("JWT_EXPIRE",  Env::get("JWT_EXPIRE",
"3600"))
```

A2.10.3 .env File — Real Credentials

```
# .env — real credentials, NEVER commit this fileDB_HOST=localhostDB_PORT=3306
DB_NAME=hiveshareDB_USER=rootDB_PASS=secret123APP_NAME=HiveShareAPP_DEBUG=false
BASE_URL=http://localhost:8080JWT_SECRET=your-secret-key-hereJWT_EXPIRE=3600
```

A2.10.4 libraries/Database.wolf — The Connection Class

This is Wolf's equivalent of your PHP PDO wrapper. It reads DB_HOST, DB_USER etc from the constants defined in config/config.wolf and wraps Go's database/sql driver in a PHP-familiar API.

```
// libraries/Database.wolf// Wolf's PDO-equivalent database class.// Mirrors the
TraversyMVC Database class exactly.class Database {    private $host   = DB_HOST
private $user   = DB_USER    private $pass   = DB_PASS    private $dbname = DB_NAME
private $port   = DB_PORT    private $dbh    # connection handle    private $stmt   #
prepared statement    private $error    func __construct() {        $dsn =
"mysql:host={$this->host};port={$this->port};dbname={$this->dbname};charset=utf8mb4"
try {            $this->dbh = Wolf\PDO::connect($dsn, $this->user, $this->pass,
[                "persistent" => true,                "errmode"    =>
"exception"            ])        } catch ($e) {            $this->error = $e->message
log("DB connection failed: {$this->error}")        }    }    # Prepare statement with
query    func query($query) {        $this->stmt = $this->dbh->prepare($query)    }    #
Bind values — auto-detects type exactly like your PHP switch(true)    func bind($param,
$value, $type = nil) {        if $type == nil {            match true
{                is_int($value)  => $type = "int"                is_bool($value) =>
$type = "bool"                is_null($value) => $type = "null"                _
=> $type = "string"            }        }        $this->stmt->bindValue($param, $value,
$type)    }    # Execute the prepared statement    func execute() {        return
$this->stmt->execute()    }    # Get result set as array of objects    func resultSet()
{        $this->execute()        return $this->stmt->fetchAll()    }    # Get single
record as object    func single() {        $this->execute()        return
$this->stmt->fetch()    }    # Get row count    func rowCount() {        return
$this->stmt->rowCount()    }    # Last inserted ID    func lastInsertId()
{        return $this->dbh->lastInsertId()    }    # Transactions    func
beginTransaction() { return $this->dbh->beginTransaction() }    func commit()
{ return $this->dbh->commit() }    func rollBack()         { return
$this->dbh->rollBack() }}
```

A2.10.5 Using the Database in a Model

Models instantiate new Database in their constructor and use $this->db-> exactly like TraversyMVC PHP. Any PHP developer reads this instantly.

```
// models/User.wolfclass User {    private $db    func __construct() {        $this->db
= new Database    }    func register($data) {        $userId =
$this->generateCustomUserId()        $this->db->query("INSERT INTO initkey_rid
(user_id, first_name, last_name, email, phone, password_hash)
VALUES (:user_id, :first_name, :last_name, :email, :phone, :password_hash)")
$this->db->bind(":user_id",       $userId)        $this->db->bind(":first_name",
$data["first_name"])        $this->db->bind(":last_name",     $data["last_name"])
$this->db->bind(":email",         $data["email"])        $this->db->bind(":phone",
$data["phone"])        $this->db->bind(":password_hash", $data["password"])        if
$this->db->execute() {            $this->assignRole($userId, "passenger")
$this->createWallet($userId)            return $userId        }        return false    }
func findUserByEmail($email) {        $this->db->query("SELECT id FROM initkey_rid WHERE
email = :email")        $this->db->bind(":email", $email)        $this->db->single()
return $this->db->rowCount() > 0    }    func findUserByEmailAndGetDetails($email)
{        $this->db->query("SELECT u.*, w.balance, w.held_funds, w.currency,
w.status as wallet_status                FROM initkey_rid u
LEFT JOIN wallets w ON u.user_id = w.user_id                WHERE u.email
= :email")        $this->db->bind(":email", $email)        return
$this->db->single()    }    func assignRole($userId, $roleName)
```

```
{        $this->db->query("SELECT id FROM roles WHERE name = :name")
$this->db->bind(":name", $roleName)        $role = $this->db->single()        if $role
{            $this->db->query("INSERT IGNORE INTO user_roles (user_id, role_id) VALUES
(:user_id, :role_id)")            $this->db->bind(":user_id",  $userId)
$this->db->bind(":role_id",  $role->id)            return $this->db->execute()        }
return false    }    func createWallet($userId) {        $this->db->query("INSERT INTO
wallets (user_id, balance, held_funds, currency, status)                        VALUES
(:user_id, 0.00, 0.00, 'NGN', 'active')")        $this->db->bind(":user_id", $userId)
return $this->db->execute()    }    func generateCustomUserId() {        return
"HiveShare" . date("M.d.Y") . time() . date("H.s") . md5(date("ymdhms") . rand(1000,
9999))    }}
```

A2.11 Python Environment — wolf.python

> *Wolf manages Python virtual environments automatically. The developer adds packages with wolf python add. wolf run handles the rest. No pip, no venv, no activation scripts.*

A2.11.1 wolf.python Config File

```
# wolf.python — Python environment for this Wolf project# Committed to git. Credentials
never go here.python_version = "3.11"venv           = ".wolf-python"    # add
to .gitignorerequirements = [    "torch>=2.0",    "transformers>=4.35",    "scikit-
learn>=1.3",    "numpy>=1.24",    "pandas>=2.0",]# Optional# requirements_file =
"requirements.txt"# extra_index_url   = "https://pypi.mycompany.com/simple"#
install_flags     = ["--no-cache-dir"]
```

A2.11.2 wolf python CLI

```
wolf python install           # install all packages from wolf.pythonwolf python add
torch         # add package + update wolf.python + lockwolf python add "numpy>=1.24" #
add with version constraintwolf python remove pandas     # remove packagewolf python
list              # list installed packages and versionswolf python check             #
verify all requirements are installedwolf python reset             # delete venv and
recreate from scratchwolf python shell             # open Python REPL inside Wolf venv
```

A2.11.3 Automatic Activation on wolf run

When wolf run or wolf build is called, Wolf automatically:

1. Detects @ml blocks in source. If none, skips Python setup entirely.
2. Checks wolf.python exists. If missing: error — No wolf.python found. Run: wolf python init
3. Checks .wolf-python venv. If missing: creates it using python_version.
4. Runs wolf python check. Installs any missing packages automatically.
5. Sets PYTHONPATH and LD_LIBRARY_PATH to .wolf-python before compilation.
6. Compiles and runs. All @ml blocks have full access to installed packages.

A2.11.4 Git Rules for wolf.python

wolf.python	COMMIT. The environment definition. Like package.json.
.wolf-python/	DO NOT COMMIT. Add to .gitignore. Auto-regenerated on each

	machine.
wolf.python.lock	COMMIT. Auto-generated. Exact installed versions for reproducible builds.

A2.11.5 wolfpkg vs Python — The Permanent Separation

> *wolfpkg (Phase 2) manages Wolf packages. wolf.python manages Python packages. Both exist permanently. wolfpkg getting mature does not remove the need for Python. @ml blocks always need Python because PyTorch, TensorFlow, and scikit-learn are Python/C libraries — they will not be rewritten in Wolf.*

A2.12 wolf new — Project Scaffolding

wolf new generates a complete project skeleton with the correct TraversyMVC folder structure. Nothing to configure manually.

```
wolf new myapp# Generated structure:myapp/  config/    config.wolf       ← pre-filled
with DB + app constants  libraries/    Database.wolf     ← full Database class ready to
use  models/            ← empty, ready for your models  controllers/        ← empty,
ready for your controllers  views/              ← empty, ready for your views
wolf.python         ← empty requirements (add packages as needed)  .env.example        ←
all keys listed, values empty  .gitignore          ← includes .env, .wolf-python,
wolf_out/  wolf.mod            ← module definition  main.wolf           ← Hello World +
App::create() starter  README.md
```

PART B — SPRINT TASK BOARDS

> *A task is DONE only when its acceptance criteria are met AND the Founder has reviewed and approved. All tasks start as TODO. Update status as work progresses: TODO → WIP → DONE.*

B1. Month 1 — Core Compiler (Weeks 1–4)

Goal: Wolf source code compiles to a working Go binary. Hello World runs. A basic HTTP endpoint responds to curl.

B1.1 Week 1 — Repository & Lexer

Task	Owner	Week	Status	Acceptance Criteria
Create GitHub repo & Makefile	Founder	**W1**	TODO	Repo created. Makefile: make build, make test, make run. Go module initialised (go mod init). cobra CLI skeleton compiles with wolf --help.
Define all token types	Claude	**W1**	TODO	token_types.go lists every token from the C1 spec table. Every token has a typed constant. File reviewed by Gemini.
Implement Lexer	Claude	**W1**	TODO	Reads UTF-8 source. Emits []Token with type, literal, line, col, file. Handles: $vars, string interpolation {$var}, @ml, #comments, multiline strings.
Lexer unit tests — full coverage	Claude	**W1**	TODO	go test ./internal/lexer/... passes. Every token type has tests. Edge cases: empty file, unicode, nested interpolation, unterminated string.
Error format standard	Claude	**W1**	TODO	All errors print: wolf: error at line 4, col 12: <message>. Format is machine-parseable. Implement this pattern once, reuse everywhere.

B1.2 Week 2 — Parser & AST

Task	Owner	Week	Status	Acceptance Criteria
Define all AST node types	Claude	**W2**	TODO	ast_nodes.go has typed structs for every construct in A2. All nodes implement the Node interface (nodeType(), Pos()). Gemini reviews for completeness.
Implement Parser	Claude	**W2**	TODO	Recursive-descent PEG parser. Correct operator precedence. Produces valid AST for all syntax in A2.1–A2.8. Error on invalid syntax with line/col.
Parser unit tests	Claude	**W2**	TODO	Every AST node type: minimum 3 passing tests, 1 error test. All example code from A2 parses without errors.
AST pretty-printer	Claude	**W2**	TODO	wolf fmt --ast source.wolf prints AST in readable indented form. Used for debugging only. Not user-facing.

Integration parse test	Claude	**W2**	TODO	Parse examples/hello/hello.wolf and examples/webserver/server.wolf without errors. AST shape verified.

B1.3 Week 3 — Resolver, Go Emitter, Hello World

Task	Owner	Week	Status	Acceptance Criteria
Implement Resolver	Claude	**W3**	TODO	Walk AST. Build scope tree. Resolve $variables to declarations. Strip $ prefix for Go output. Error on undeclared variable. Attach symbol table to AST nodes.
Implement Go Emitter — core	Claude	**W3**	TODO	Emit valid gofmt-compatible Go for: variables, all control flow, functions, classes→structs+methods, string interpolation→fmt.Sprintf, arrays→[]any, maps→map[string]any.
wolf run and wolf build CLI	Claude	**W3**	TODO	wolf run hello.wolf: compiles to temp dir, runs binary. wolf build hello.wolf -o hello: produces named binary. Both work end-to-end.
MILESTONE: Hello World	Founder	**W3**	TODO	examples/hello/hello.wolf: print("Hello from Wolf!"). wolf run hello.wolf prints the string. Founder demos this. Project is not blocked after this.
wolf fmt command	Claude	**W3**	TODO	wolf fmt source.wolf reformats Wolf source consistently. Idempotent — running twice produces same result.

B1.4 Week 4 — HTTP Server & First Web Endpoint

Task	Owner	Week	Status	Acceptance Criteria
Wolf stdlib: http module	Claude	**W4**	TODO	Wolf\App::create(), ->get/post/put/delete/use/group/serve() all work. Routes support {param}. Wraps net/http. Tested with unit tests.
Request / Response objects	Claude	**W4**	TODO	$req: ->json(), ->body(), ->param(), ->query(), ->header(), ->auth(). $res: ->json(), ->text(), ->html(), ->error(), ->redirect(). All tested.
Async blocks + goroutines	Claude	**W4**	TODO	async {} → goroutine. await → channel receive. parallel {} → sync.WaitGroup. All three work correctly. Race detector (go test -race) passes.
Error handling	Claude	**W4**	TODO	try/catch compiles to if err != nil. Multiple return [$val, $err] works. Wolf\Error type implemented and tested.
MILESTONE: HTTP API	Founder	**W4**	TODO	examples/webserver/server.wolf: GET / returns {status:ok}, POST /echo returns body, GET /users/:id returns {id:x}. curl tests pass for all three routes.

B2. Month 2 — Python Bridge & Database (Weeks 5–8)

Goal: @ml blocks work with real PyTorch. MySQL, Redis, MongoDB connect via config files. Showcase demo app runs end-to-end.

B2.1 Week 5 — wolf.python & CPython Bridge Foundation

Task	Owner	Week	Status	Acceptance Criteria
wolf.python parser	Claude	**W5**	TODO	Parse wolf.python config. Return WolfPythonConfig struct with all fields. Clear error on malformed config. Unit tested.
wolf python install	Claude	**W5**	TODO	Creates .wolf-python venv using python_version. Runs pip install for each requirement. Writes wolf.python.lock with exact versions. Works on Linux and macOS.
wolf python add/remove/list/check	Claude	**W5**	TODO	add: installs + updates wolf.python + lock. remove: uninstalls + updates. list: prints package versions. check: fails clearly if any requirement missing.
Auto venv activation in wolf run	Claude	**W5**	TODO	wolf run: detects @ml, checks wolf.python, auto-creates venv if needed, auto-installs missing packages, sets PYTHONPATH+LD_LIBRARY_PATH. Tested end-to-end.
cgo + libpython setup	Claude	**W5**	TODO	cgo_bridge.c initialises CPython (Py_Initialize), evaluates Python source, handles Py_Finalize. Links against .wolf-python/lib/libpython3.11. Compiles on Linux + macOS.
Wolf→Python type marshalling	Claude	**W5**	TODO	All types in A2.8.1 bridge table: Go→Python conversion correct. 50+ unit tests. Every type combination covered.
Python→Wolf type unmarshalling	Claude	**W5**	TODO	PyLong, PyFloat, PyUnicode, PyBool, PyList, PyDict, PyBytes all unmarshal correctly to Go types. Type errors return descriptive Wolf errors.
GIL management	Claude	**W5**	TODO	Goroutines acquire GIL via PyGILState_Ensure, release via PyGILState_Release. 100 concurrent @ml goroutines do not deadlock. Stress test passes.
Python init once	Claude	**W5**	TODO	CPython initialised exactly once via sync.Once. Startup overhead <200ms measured and logged.

B2.2 Week 6 — @ml Compiler Support

Task	Owner	Week	Status	Acceptance Criteria
@ml lexer tokens	Claude	**W6**	TODO	AT_ML token emitted. Python code inside @ml {} preserved verbatim as string — not re-lexed as Wolf. @ml model() and @ml async variants handled.
MLBlock AST node	Claude	**W6**	TODO	MLBlock contains: python_source, in_vars [], out_vars [], model_name (optional), is_async. Fully parsed from all @ml variants.
@ml Go emitter	Claude	**W6**	TODO	@ml block compiles to: (1) marshal in_vars, (2) call mlbridge.Execute(), (3) unmarshal out_vars. Generated Go is readable. Unit tested.
@ml variable	Claude	**W6**	TODO	Wolf vars before @ml available in Python (without $). Python vars assigned inside available in Wolf after.

scoping				Shadowing handled correctly.
MILESTONE: PyTorch in Wolf	Founder	**W6**	TODO	Wolf file: imports torch in @ml, creates tensor, runs operation, returns to Wolf scope, prints. Must run with wolf run. Founder verifies output.

B2.3 Week 7 — Database Library & Drivers

Task	Owner	Week	Status	Acceptance Criteria
config/config.wolf parser	Claude	**W7**	TODO	Wolf compiler reads config/config.wolf at startup. define() calls register named constants available anywhere in the app. Env::get() resolves from .env. Missing .env key with no default = clear error.
Wolf\PDO driver wrapper	Claude	**W7**	TODO	Wolf\PDO::connect() wraps Go database/sql + go-sql-driver/mysql. Returns handle with: prepare(), bindValue(), execute(), fetchAll(), fetch(), rowCount(), lastInsertId(), beginTransaction(), commit(), rollBack(). All tested.
libraries/Database.wolf class	Claude	**W7**	TODO	Implement exact class from A2.10.4. Reads DB_HOST/USER/PASS/NAME constants. query(), bind() with auto-type detection, execute(), resultSet(), single(), rowCount(), lastInsertId(), beginTransaction(), commit(), rollBack(). All methods tested.
models/User.wolf example	Claude	**W7**	TODO	Implement the full User model from A2.10.5 using new Database + $this->db->. register(), findUserByEmail(), findUserByEmailAndGetDetails(), assignRole(), createWallet(), generateCustomUserId() all work correctly.
Redis library class	Claude	**W7**	TODO	libraries/Redis.wolf: connects using REDIS_HOST/PORT/PASS constants. get(), set(ttl:), del(), exists(), expire(), hget(), hset(), lpush(), lrange(). Same $this->redis-> pattern as Database.
MongoDB library class	Claude	**W7**	TODO	libraries/MongoDB.wolf: connects using MONGO_HOST/PORT/DB constants. collection()->find(), findOne(), insertOne(), updateOne(), deleteOne(). Same $this->mongo-> pattern.
DB integration tests	Claude	**W7**	TODO	Docker Compose: MySQL + Redis + MongoDB. Integration tests compile and run a Wolf app that uses all three libraries. All CRUD operations pass. CI runs these on every PR.

B2.4 Week 8 — Showcase Demo App

Task	Owner	Week	Status	Acceptance Criteria
Design demo architecture	Founder	**W8**	TODO	Ride-pricing API: POST /rides/estimate — MySQL for driver data, Redis caching, @ml for surge model. Founder approves schema and flow before Claude builds.
wolf new + config setup	Claude	**W8**	TODO	Run wolf new surge-demo. Fill config/config.wolf constants. Create .env with real values. Create libraries/Database.wolf. Add scikit-learn to wolf.python.

				Verify wolf python check passes.
Demo: database layer	Claude	**W8**	TODO	schema.sql with drivers and ride_zones tables. Seed data. Wolf model uses new Database + $this->db->query/bind to query drivers by zone. Unit tested.
Demo: @ml surge model	Claude	**W8**	TODO	scikit-learn LinearRegression trained on synthetic data. Saved as surge_v1.pkl. @ml block loads model, predicts multiplier from driver count + time of day.
Demo: HTTP layer	Claude	**W8**	TODO	POST /rides/estimate: validates input, queries DB, checks Redis cache, runs @ml, caches result, returns price. GET /health. Full error handling.
MILESTONE: Full demo runs	Founder	**W8**	TODO	curl POST /rides/estimate returns JSON price. Response <200ms. 1000 concurrent requests without crash. Founder records screen demo video (this becomes the launch asset).

B3. Month 3 — Polish, Docs & Launch (Weeks 9–12)

Goal: Wolf is publicly releasable. Docs are complete. GitHub is live. Launch executed.

B3.1 Week 9 — Standard Library & Error Quality

Task	Owner	Week	Status	Acceptance Criteria
Wolf stdlib: strings	Claude	**W9**	TODO	Wolf\Strings: contains, startsWith, endsWith, split, join, trim, upper, lower, replace, length, repeat, pad. All tested.
Wolf stdlib: json	Claude	**W9**	TODO	Wolf\JSON: encode, decode, encodePretty. Nested objects, arrays, null. Error on invalid JSON.
Wolf stdlib: math	Claude	**W9**	TODO	Wolf\Math: abs, ceil, floor, round, sqrt, pow, log, min, max, random, pi. All tested.
Wolf stdlib: env	Claude	**W9**	TODO	Wolf\Env: get($key, $default), set, all(), loadFile(".env"). .env format supported.
wolf new command	Claude	**W9**	TODO	wolf new myapp generates full project skeleton from A2.12 spec. All config templates filled. .gitignore correct. wolf run main.wolf works immediately.
Error quality overhaul	Claude	**W9**	TODO	Every error: (1) the source line, (2) caret pointing to problem, (3) helpful message with suggestion. Model: Rust/Elm error quality. No raw panics on any user input — ever.

B3.2 Week 10 — Documentation

Task	Owner	Week	Status	Acceptance Criteria
Getting Started guide	Claude	**W10**	TODO	docs/getting_started.md: install Wolf, Hello World, first HTTP server, first @ml block. Readable in 15 minutes.

				Every example tested and working.
Language reference	Claude	**W10**	TODO	docs/syntax.md: complete spec for every feature. Every section has a working example. Covers all A2 sections.
Database guide	Claude	**W10**	TODO	docs/database.md: config/config.wolf constants format, .env setup, libraries/Database.wolf full API, model usage with $this->db->, Redis and MongoDB library classes. Docker Compose for local dev.
ML bridge + wolf.python guide	Claude	**W10**	TODO	docs/ml_bridge.md + docs/python_env.md: @ml syntax, type bridge table, wolf.python format, wolf python CLI, PyTorch/sklearn/HuggingFace examples, CI setup.
README.md	Founder	**W10**	TODO	GitHub README. Wolf logo, one-liner, code teaser (20 lines of the surge pricing demo), install instructions, links to docs, demo video. Founder writes this — it is the first impression.

B3.3 Week 11 — Cross-Platform Builds & CI

Task	Owner	Week	Status	Acceptance Criteria
Linux amd64 binary	Claude	**W11**	TODO	wolf binary for Linux x86-64. Released as wolf-linux-amd64. Install: download, chmod +x, move to /usr/local/bin. Tested on Ubuntu 22.04.
macOS arm64 binary	Claude	**W11**	TODO	wolf binary for macOS Apple Silicon. Released as wolf-darwin-arm64. Tested on M2 Mac.
macOS amd64 binary	Claude	**W11**	TODO	wolf binary for macOS Intel. Released as wolf-darwin-amd64.
GitHub Actions CI	Claude	**W11**	TODO	On PR: unit tests, integration tests (Docker), example compilation, benchmarks. PR blocked if any fail. On merge to main: build all three binaries, publish to GitHub Releases.
End-to-end test suite	Claude	**W11**	TODO	20 Wolf programs covering all features. All pass on all three platforms. Includes: variables, functions, classes, HTTP, @ml, DB queries, concurrency, error handling.

B3.4 Week 12 — Launch Week

Task	Owner	Week	Status	Acceptance Criteria
GitHub repo public	Founder	**W12**	TODO	Make repo public. Apache 2.0 license. CONTRIBUTING.md written. Good first issue pinned for community.
GitHub Releases v0.1.0	Claude	**W12**	TODO	Tag v0.1.0. All three platform binaries attached. Changelog written. Release notes explain what works and what is Phase 2.
Hacker News Show HN	Founder	**W12**	TODO	Post: Show HN: Wolf — PHP syntax + Go performance + Python ML in one language. Include code demo, GitHub link, honest scope description. Post Tuesday 09:00 ET.
Dev.to launch article	Founder	**W12**	TODO	Why I built a programming language with Claude and Gemini as co-engineers. Personal story, code

				examples. Published same day as HN post.
Demo video	Founder	**W12**	**TODO**	3–5 minute screen recording. Install Wolf, write the surge pricing demo from scratch, wolf run it, hit it with curl. Upload to YouTube. Link from README.
Respond to all HN comments	Founder	**W12**	**TODO**	Engage with every comment on launch day. Traction comes from conversation, not just upvotes. This is the Founder's most important task on launch day.

PART C — CODE SPECIFICATION

C1. Complete Token Type List

The lexer must produce exactly these tokens. Any sequence not mappable to one of these is a lexer error.

Token	Example	Notes
DOLLAR_IDENT	$myVar	$ + [a-zA-Z_][a-zA-Z0-9_]*
STRING	"hello"	Double-quoted. Contains interp segments.
STRING_INTERP	"{$name}"	Emitted as START, DOLLAR_IDENT, END segments
INT	42, -7	64-bit signed integer literal
FLOAT	3.14, -0.5	64-bit float literal
BOOL	true, false	Lowercase only
NIL	nil	Null value
FUNC	func	Function keyword
CLASS	class	Class definition
NEW	new	Object instantiation
IF / ELSE	if, else	Conditional
FOR	for	C-style for loop
FOREACH / AS	foreach, as	Iteration
WHILE	while	While loop
MATCH	match	Pattern match / switch
RETURN	return	Return statement
ASYNC	async	Async block
AWAIT	await	Await expression
PARALLEL	parallel	Parallel block
AT_ML	@ml	ML block — followed by {, model(), or async
TRY / CATCH	try, catch	Error handling sugar
VAR	var	Typed declaration
ARROW	=>	Match arm / arrow function
FAT_ARROW	->	Property/method access
DOUBLE_COLON	::	Static method call
LBRACE/RBRACE	{ }	Block delimiters
LPAREN/RPAREN	( )	Grouping / params
LBRACKET/RBRACKET	[ ]	Array access / literals

COMMA	,	Separator
COLON	:	Map key, type annotation, named param
HASH	#	Comment — rest of line ignored
PLUS/MINUS/STAR/SLASH/PERCENT	+ - * / %	Arithmetic
EQ/NEQ/LT/GT/LTE/GTE	== != < > <= >=	Comparison
AND/OR/NOT	&& || !	Logical
ASSIGN	=	Assignment
COMPOUND_ASSIGN	+= -= *= /=	Compound assignment
INCR/DECR	++ --	Increment/decrement
EOF	(end of file)	Signals end of token stream

C2. Core Go Data Structures

These are the exact structs Claude will implement. Gemini reviews all struct designs before coding begins.

C2.1 Token

```
// internal/lexer/token.gotype TokenType stringtype Token struct {    Type    TokenType
Literal string    // exact source text    Line    int    Col     int    File    string}
```

C2.2 Core AST Nodes

```
// internal/parser/ast_nodes.gotype Node interface { nodeType() string; Pos() Position }
type Stmt interface { Node; stmtNode() }type Expr interface { Node; exprNode() }type
Position struct { Line, Col int; File string }type ProgramNode   struct { Stmts []Stmt;
Pos_ Position }type VarAssignStmt struct { Name string; Value Expr; TypeAnn string; Pos_
Position }type FuncDecl      struct { Name string; Params []Param; ReturnTypes []string;
Body []Stmt; IsArrow bool; Pos_ Position }type ClassDecl     struct { Name string;
Fields []Field; Methods []FuncDecl; Pos_ Position }type IfStmt        struct { Cond
Expr; Then []Stmt; ElseIf []ElseIfClause; Else []Stmt; Pos_ Position }type ForStmt
struct { Init Stmt; Cond Expr; Post Stmt; Body []Stmt; Pos_ Position }type ForEachStmt
struct { Key, Value string; Iter Expr; Body []Stmt; Pos_ Position }type WhileStmt
struct { Cond Expr; Body []Stmt; Pos_ Position }type MatchStmt     struct { Subject
Expr; Arms []MatchArm; Pos_ Position }type AsyncBlock    struct { Body []Stmt; IsAwait
bool; Pos_ Position }type ParallelBlock struct { Assignments []VarAssignStmt; Pos_
Position }type TryCatch      struct { Try []Stmt; ErrVar string; Catch []Stmt; Pos_
Position }type MLBlock struct {    PythonSrc  string    InVars     []string  // explicit
in vars, empty = auto-detect from scope    OutVars    []string  // explicit out vars
ModelName  string    // optional cached model name    IsAsync    bool    Pos_
Position}
```

C2.3 Database Library — Go Implementation Contract

This is what Claude will implement in Go to back libraries/database.wolf. The Wolf Database class compiles down to these Go types.

```
// internal/stdlib/database.go// WolfDB wraps Go's database/sql with a PHP/PDO-familiar
API.// One instance per model — created by new Database in Wolf.type WolfDB struct
{    db    *sql.DB    stmt *sql.Stmt    rows *sql.Rows    err  error}// NewWolfDB reads
DB_HOST/USER/PASS/NAME/PORT from Wolf constants// (resolved from config/config.wolf at
compile time) and opens// a persistent connection pool.func NewWolfDB() (*WolfDB, error)
// Query prepares a statement. Mirrors PHP ->query()func (w *WolfDB) Query(sql string)
error// Bind binds a named parameter with auto type detection.// Mirrors PHP
->bind(':param', $value)func (w *WolfDB) Bind(param string, value any) error// Execute
runs the prepared statement. Returns bool.func (w *WolfDB) Execute() bool// ResultSet
executes and returns all rows as []map[string]any.// Each map is one row — keys are
column names.func (w *WolfDB) ResultSet() []map[string]any// Single executes and returns
one row as map[string]any.func (w *WolfDB) Single() map[string]any// RowCount returns
the number of rows affected/returned.func (w *WolfDB) RowCount() int// LastInsertId
returns the last auto-increment ID.func (w *WolfDB) LastInsertId() int64//
BeginTransaction, Commit, RollBack mirror PHP PDO transactions.func (w *WolfDB)
BeginTransaction() errorfunc (w *WolfDB) Commit() errorfunc (w *WolfDB) RollBack() error
// WolfRedis wraps go-redis with PHP-familiar method names.type WolfRedis struct
{ client *redis.Client }func NewWolfRedis() (*WolfRedis, error)  // reads
REDIS_HOST/PORT/PASSfunc (r *WolfRedis) Get(key string) (string, error)func (r
*WolfRedis) Set(key string, value any, ttl int) errorfunc (r *WolfRedis) Del(key string)
errorfunc (r *WolfRedis) Exists(key string) bool// WolfMongo wraps mongo-driver with
PHP-familiar method names.type WolfMongo struct { client *mongo.Client; db
*mongo.Database }func NewWolfMongo() (*WolfMongo, error)  // reads MONGO_HOST/PORT/DB
func (m *WolfMongo) Collection(name string) *WolfCollection
```

C2.4 WolfPythonConfig

```
// internal/pythonenv/config.gotype WolfPythonConfig struct {    PythonVersion    string
VenvPath         string    Requirements     []string    RequirementsFile string
ExtraIndexURL    string    InstallFlags     []string}type WolfPythonLock struct
{    GeneratedAt   time.Time    PythonVersion string    Packages      []LockedPackage}
type LockedPackage struct {    Name    string   // e.g. "torch"    Version string   //
exact e.g. "2.1.0"    Hash    string   // sha256 for integrity}type PythonEnvManager
interface {    ParseConfig(projectRoot string) (*WolfPythonConfig, error)
EnsureVenv(cfg *WolfPythonConfig, projectRoot string) error    InstallRequirements(cfg
*WolfPythonConfig, projectRoot string) error    Check(cfg *WolfPythonConfig, projectRoot
string) error    Activate(cfg *WolfPythonConfig, projectRoot string) error
WriteLock(cfg *WolfPythonConfig, projectRoot string) error}
```

C2.5 ML Bridge Interface

```
// internal/mlbridge/bridge.gotype MLBridge interface {    // Init initialises CPython.
Call exactly once at startup.    Init(pythonPath string) error    // Execute runs
pythonSrc with inVars as Python locals.    // Returns outVars extracted from Python
locals after execution.    Execute(pythonSrc string, inVars map[string]any)
(map[string]any, error)    // ExecuteAsync runs Execute in a goroutine. Returns a result
channel.    ExecuteAsync(pythonSrc string, inVars map[string]any) <-chan MLResult    //
LoadModel loads and caches a model by name.    LoadModel(name string, loadScript string)
error    // Shutdown cleanly finalises CPython. Call at program exit.    Shutdown()}type
MLResult struct {    Vars map[string]any    Err  error}
```

PART D — TESTING & LAUNCH

D1. Testing Strategy

> *A compiler with untested output is a liability. Every component has tests before it is done. The AI team writes excellent tests — require them. No task marked DONE without passing tests.*

D1.1 Test Pyramid

Unit tests	Every function in every package. Target 90%+ coverage. Go testing + testify. Run on every commit.
Integration tests	Compile and run real Wolf programs end-to-end. 20+ programs. Require Docker for DB tests. Run on every PR.
Performance tests	Benchmark: lexer throughput, parse speed, emit speed, DB query latency, @ml execution time. Regression blocks merge.
Stress tests	@ml under 100 concurrent goroutines. HTTP under 10,000 concurrent connections. DB pool exhaustion handling.
Negative tests	50+ invalid Wolf programs. All must produce helpful errors. Zero panics on invalid input — ever.
Config tests	Missing config/config.wolf, malformed constants, missing .env keys — all produce clear, actionable error messages.

D1.2 The Five Mandatory Demos

These five programs must run before any public launch. They are the project's acceptance test.

7. **Demo 1** Hello World: print("Hello from Wolf!") compiles and prints.
8. **Demo 2** HTTP API: GET /users/:id from MySQL via libraries/Database.wolf + $this->db->. POST /users creates record. Constants from config/config.wolf.
9. **Demo 3** Concurrency: 1,000 concurrent requests, all respond in under 100ms.
10. **Demo 4** ML Bridge: POST /sentiment — @ml runs HuggingFace sentiment analysis. Returns {label, score}.
11. **Demo 5** Full Stack: Surge pricing demo — libraries/Database.wolf for MySQL, libraries/Redis.wolf for caching, @ml for model, HTTP API. config/config.wolf + .env for all credentials. All in Wolf.

D2. Launch Checklist

D2.1 Pre-Launch (Before Making Repo Public)

- All 5 mandatory demos pass on all three platforms
- README.md written, code-reviewed by Founder
- Getting Started guide tested by someone reading it for the first time
- GitHub Actions CI green on main branch
- Linux, macOS arm64, macOS amd64 binaries attached to v0.1.0 release
- Zero panics on any user input — tested with invalid code fuzzing
- Apache 2.0 LICENSE file added
- CONTRIBUTING.md written — how to file issues, how to submit PRs
- 5 example projects in examples/ directory
- .env.example committed — all keys documented, values empty

D2.2 Launch Day Sequence

12. **08:00** Publish GitHub Releases v0.1.0 — binaries live
13. **08:05** Make repository public
14. **08:30** Publish Dev.to article
15. **09:00 ET — Tuesday only** Post Hacker News Show HN
16. **09:30** Post in r/programming, r/golang, r/PHP, r/Python
17. **10:00** Share demo video link everywhere
18. **All day — Founder priority** Respond to every HN comment the same day

D3. Phase 2 — After the 3-Month Sprint

If the launch generates traction, these are the Phase 2 priorities. The Founder decides order based on community feedback.

- **Priority 1:** wolfpkg — Wolf package manager and registry. The single most requested feature will be third-party packages.
- **Priority 2:** WolfMVC — Laravel/TraversyMVC-inspired framework. Models, Views, Controllers, routing, validation, ORM. This is what brings PHP developers in volume.
- **Priority 3:** VS Code extension — syntax highlighting and language server. Developers ask for this immediately after any language launch.
- **Priority 4:** Windows support — builds for Windows amd64.
- **Priority 5:** Performance benchmarks published — official Wolf vs Go vs PHP vs Python numbers. This drives press coverage.
- **Priority 6:** Wolf Cloud — a hosting platform specifically for Wolf apps. Vercel for Wolf. Long-term business model.

B *v3.1*

Now